

■ NutriGuide — Complete Interview Preparation Guide

Use this as your script. Practice saying these answers out loud before the interview.

■ STEP 1 — Project Introduction (Say This First)

***"Tell me about your project."**

Say this:

"I built NutriGuide — a full-stack personalized Indian diet and nutrition tracking web application. The idea is simple: most diet apps give generic recommendations, but NutriGuide gives personalized Indian meal suggestions based on each user's health goal, diet preference, and allergies.*

The app has features like:

- *User registration and login with JWT-based authentication*
 - *Personalized recipe recommendations (lose weight / gain muscle / maintain)*
 - *Daily meal logging — breakfast, lunch, dinner, snack*
 - *Weekly nutrition analytics with charts (calories, protein, carbs, fat)*
 - *Profile management — users update their weight, allergies, goals*
 - *An Admin Panel for managing recipes*
-

■■ STEP 2 — Tech Stack (What & Why)

***"What tech stack did you use and why?"**

Say this:

"I used a MERN-style stack but with MySQL instead of MongoDB, because nutrition data is highly relational — users, recipes, and meal logs have clear relationships between them, so a relational database made more sense."*

■■ STEP 3 — Architecture Explanation

***"Explain the architecture of your project."**

Say this:

"NutriGuide follows a classic 3-tier architecture:"*

```
TIER 1: PRESENTATION    →  React Frontend  (localhost:5173)
TIER 2: BUSINESS LOGIC  →  Express Backend (localhost:5000)
```

TIER 3: DATA STORAGE → MySQL Database (localhost:3306)

"The frontend and backend communicate over HTTP using REST APIs. The backend and database communicate using SQL queries through the mysql2 connection pool. The whole system is secured using JWT tokens."

Backend Folder Structure — How to Explain It

```
backend/
  ■■■ server.js      → Entry point. Starts Express, mounts all routes
  ■■■ config/db.js   → MySQL connection pool (shared across all controllers)
  ■■■ middlewares/   → Security guards (verifyToken, isAdmin)
  ■■■ routes/        → URL maps – which URL calls which controller function
  ■■■ controllers/   → Business logic – actual code that runs for each request
```

"I followed the MVC pattern — Models (MySQL tables), Views (React pages), Controllers (Express controller files). This keeps concerns separated and code maintainable."

■ STEP 4 — Authentication (Deep Explanation)

"How does authentication work in your project?"

Say this:

"I implemented JWT-based stateless authentication. Let me walk you through the full flow:"

Registration

1. User submits form: name, email, password, age, weight, health_goal
2. Backend checks if email already exists in DB
3. Password is hashed using bcrypt with salt rounds of 10
 - `bcrypt.hash("mypassword", 10) → "$2b$10$kh9X..."`
 - Even if DB is hacked, passwords are not readable
4. User row is inserted into MySQL `users` table
5. Response: `{ message: "User registered", userId: 1 }`

Login

1. User sends: `{ email, password }`
2. Backend fetches user from DB by email
3. `bcrypt.compare(inputPassword, hashedPassword) → true/false`
4. If true: `jwt.sign({ id: 1, role: 'user' }, SECRET, { expiresIn: '24h' })`
5. Token + user data returned to frontend
6. Frontend stores token in `localStorage`
7. `AuthContext` sets global user state

Every Protected Request

1. Axios interceptor reads token from `localStorage`
2. Adds Authorization: "Bearer eyJhbGc..." to every request header
3. Backend `verifyToken` middleware:
 - Reads the header
 - `jwt.verify(token, SECRET) → decodes { id: 1, role: 'user' }`
 - Sets `req.userId = 1, req.userRole = 'user'`
 - Calls `next()` → controller runs
4. Controller uses `req.userId` to query the right user's data

Key Concept — Why JWT?

"JWT is stateless — the server doesn't store sessions. The token itself contains the user's ID and role. The server just verifies the signature using the secret key. This makes it scalable — any server instance can verify any token without sharing session storage."

Two Middleware Guards

```
verifyToken → "Are you logged in?" (checks JWT)
isAdmin     → "Are you an admin?" (checks role in decoded token)
```

■ STEP 5 — CRUD Operations (All 3 Modules)

****"Explain the CRUD operations in your project."****

Module 1: User / Profile

Module 2: Recipes

Module 3: Meal Logs

■ STEP 6 — Smart Recommendation Algorithm

****"How do personalized recommendations work?"****

****Say this:****

"This was the most interesting part to build. The algorithm has 3 stages:"

STAGE 1 – FILTER by diet preference

- vegan → only vegan recipes
- vegetarian → vegetarian + small % of vegan
- non-veg → 70% veg + 30% non-veg (healthy balance)

STAGE 2 – FILTER by allergies

- User's allergies stored as comma-separated string in DB
e.g., "nuts,dairy"
- Each recipe's ingredients (JSON array) is checked
- Any recipe containing an allergen is removed

STAGE 3 – SORT by health goal

- lose_weight → filter calories ≤ 500, sort lowest first
- gain_muscle → filter calories ≥ 600, sort highest first
- maintain → random shuffle

RESULT → Return top 10 recipes

■■ STEP 7 — Database Design

****"Explain your database design."****

****Say this:****

"The database has 3 tables with relationships between them:"

`users` table

```
Stores: id, name, email, hashed_password, role (user/admin),
        age, weight, height, diet_preference, health_goal, allergies, target_weight
Key constraint: email must be UNIQUE
```

`recipes` table

Stores: id, title, description, ingredients (JSON), instructions, calories, protein, carbs, fat, diet_tag, image_url, created_by (FK→users)
Key: ingredients stored as JSON array ["dal", "ghee", "turmeric"]

`meal\_logs` table

Stores: id, user_id (FK→users), recipe_id (FK→recipes), date, meal_type
This is a JUNCTION TABLE – connects users to recipes they ate

Relationships

users **1**< meal_logs **>1** recipes
(one user has many meal logs, each log links to one recipe)

users **1**< recipes
(admin users can create recipes – created_by FK)

Foreign Key Behavior

meal_logs.user_id → ON DELETE CASCADE (user deleted → logs deleted)
meal_logs.recipe_id → ON DELETE CASCADE (recipe deleted → logs deleted)
recipes.created_by → ON DELETE SET NULL (admin deleted → recipe stays, created_by = NULL)

■ STEP 8 — CORS Explanation

***"What is CORS and why did you use it?"**

Say this:

"CORS stands for Cross-Origin Resource Sharing. The browser has a built-in security rule called Same-Origin Policy — it blocks requests between different origins. My React frontend runs on port 5173 and my Express backend runs on port 5000 — these are two different origins.

Without CORS, the browser would block every API call from the frontend. I added the cors() middleware in server.js to tell the browser: 'This server allows cross-origin requests.' The backend adds specific response headers that the browser reads before allowing the response through.

Currently I have `origin: '\*'` which allows any frontend — that's fine for development. In production, I would restrict it to only my deployed frontend domain.

```
app.use(cors({
  origin: '*', // allow all origins (dev only)
  methods: ['GET', 'POST', 'PUT', 'DELETE'], // only these HTTP methods
  allowedHeaders: ['Content-Type', 'Authorization'] // only these headers
}));
```

■■ STEP 9 — Frontend Workflow

***"How is the frontend structured?"**

Say this:

"The frontend is a React SPA (Single Page Application) built with Vite. It has 4 main layers:"

1. ENTRY: main.jsx → mounts App into the DOM
2. ROUTING: App.jsx → defines all routes + route guards
3. STATE: AuthContext.jsx → global user state (who is logged in)
4. API: services/api.js → all HTTP calls in one place (Axios instance)

Route Guards (Access Control)

ProtectedRoute → must be logged in, else redirect to /
AuthRedirect → if already logged in, can't visit /login again
HomeRedirect → logged-in → /dashboard | guest → LandingPage
requireAdmin → must be logged in AND role='admin'

How AuthContext Works on Page Load

App loads → AuthContext checks localStorage for token
→ If token found → GET /api/auth/profile (verify token is still valid)
→ If valid → setUser(data) → user is auto-logged in
→ If invalid → clear token → show landing page

■ STEP 10 — Analytics Feature

How does the analytics/charts feature work?

Say this:

The Analytics page uses Recharts — a React charting library. The data comes from a single backend query that aggregates meal logs by date:

```
SELECT date,
       SUM(calories) as total_calories,
       SUM(protein)  as total_protein,
       SUM(carbs)    as total_carbs,
       SUM(fat)      as total_fat
FROM meal_logs
JOIN recipes ON meal_logs.recipe_id = recipes.id
WHERE user_id = 1
  AND date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)
GROUP BY date
ORDER BY date ASC
```

This gives one row per day with total nutrition. The frontend passes this array to Recharts LineChart/BarChart components which render it as interactive graphs.

■ EXPECTED INTERVIEW QUESTIONS + ANSWERS

Q1: What is JWT and how does it work?

JWT stands for JSON Web Token. It has 3 parts separated by dots: Header.Payload.Signature.

- Header: algorithm used (HS256)
- Payload: data stored (userId, role, expiry)
- Signature: Header + Payload encrypted with a secret key*

*The server creates it at login and gives it to the client. The client sends it back with every request. The server verifies the signature to confirm it wasn't tampered with — without hitting the database. It's stateless.***

Q2: What is bcrypt and why use it for passwords?

"bcrypt is a password hashing function. When a user registers, instead of saving their plain password, I run `bcrypt.hash(password, 10)` which generates an irreversible hash like ``$2b$10$Kh9X...``. The number 10 is the salt rounds — higher means slower but more secure.

When they log in, I use `bcrypt.compare(inputPassword, storedHash)` — it re-hashes the input and checks if they match. You can never reverse-engineer the original password from the hash. Even if someone steals the database, they can't see anyone's password."

Q3: What is the difference between authentication and authorization?

"Authentication is verifying WHO you are — login with email/password, get a JWT token. That's done by `verifyToken` middleware.

Authorization is verifying WHAT you're allowed to do — even if you're logged in, you can't access the admin panel unless your role is 'admin'. That's done by the `isAdmin` middleware."

Q4: Why MySQL instead of MongoDB?

"NutriGuide data is highly relational. A meal log links a specific user to a specific recipe on a specific date. Using foreign keys ensures data integrity — you can't log a meal for a user or recipe that doesn't exist. MySQL enforces these constraints at the database level. MongoDB would have been fine too, but MySQL was the better fit here because of these relationships."

Q5: What is a connection pool and why use it?

"A connection pool maintains a set of reusable database connections (in our case, up to 10). Instead of creating a new connection for every request — which is expensive — the pool lends an existing connection and reclaims it when done. This makes the backend much faster under concurrent user load."

Q6: What is middleware in Express?

"Middleware is a function that sits between the request arriving and the controller running. It receives `(req, res, next)` and can modify the request, terminate it with an error, or call `next()` to pass control forward.

In NutriGuide, `verifyToken` is middleware — it checks the JWT on every protected route before the controller runs. `cors()` is also middleware — it adds response headers on every response."

Q7: What is CORS and why do you need it?

*"Cross-Origin Resource Sharing. Browsers block HTTP requests between different origins (different domain/port). My React frontend (port 5173) and Express backend (port 5000) are different origins. Without `cors()` middleware adding the right response headers, the browser would block all API calls. `cors()` in the backend tells the browser 'I allow requests from other origins.'"

Q8: What is React Context and why did you use it?

"React Context is a way to share state across the entire component tree without prop drilling — passing data through every parent component manually. I used `AuthContext` to store the logged-in user's data globally. Any component in the app can call `useContext(AuthContext)` to get the current user, call `login()`, `logout()`, or `updateUser()` without receiving it as a prop."

Q9: What is an Axios interceptor?

"An interceptor runs before every request is sent or before every response is received. In NutriGuide, the request interceptor automatically reads the JWT token from localStorage and attaches it as the Authorization header to every outgoing request. This means I never have to manually add the token in Login.jsx, Meals.jsx, Dashboard.jsx, etc. — it's handled in one central place."

Q10: What is the difference between GET, POST, PUT, DELETE?

"These are HTTP methods that indicate the intent of a request:"

- GET → Read data (no body, doesn't change anything)
- POST → Create new data
- PUT → Update existing data (full or partial replace)
- DELETE → Remove data*

In NutriGuide: GET /recipes/ reads all recipes, POST /meals/ creates a new meal log, PUT /auth/profile updates the user profile, DELETE /meals/daily resets today's meal logs."

Q11: What is a Foreign Key?

*"A foreign key is a column in one table that references the primary key of another table. It creates a link between tables and enforces referential integrity — you can't insert a meal_log with user_id=999 if user 999 doesn't exist in the users table. MySQL will reject it. In NutriGuide, meal_logs has user_id (FK→users) and recipe_id (FK→recipes)."

Q12: What is a REST API?

"REST stands for Representational State Transfer. It's a set of rules for designing APIs using HTTP. A RESTful API uses standard HTTP methods (GET/POST/PUT/DELETE), stateless communication (no server-side sessions), and resource-based URLs. For example: GET /api/recipes/ to get all recipes, DELETE /api/recipes/5 to delete recipe with id 5."

Q13: What is the difference between localStorage and cookies?

*"localStorage stores data in the browser that persists even after the tab/browser is closed, until explicitly cleared. Cookies can also be persistent but are automatically sent with every HTTP request by the browser, and can be set as httpOnly (not accessible by JavaScript). I used localStorage for the JWT token for simplicity. In a more secure production app, httpOnly cookies would be safer as they're not accessible to JavaScript (preventing XSS attacks)."

Q14: How do you handle errors in your backend?

"Every controller function is wrapped in a try-catch block. If the database query fails or any unexpected error occurs, the catch block sends a 500 status with the error message: `res.status(500).json({ message: 'Server error', error: error.message })`. For specific cases like 'user not found', I return a 404. For unauthorized access, 401. For forbidden (wrong role), 403."

Q15: What security measures did you implement?

"Several:"

1. *Password hashing — bcrypt with salt rounds*
2. *JWT for stateless, signed authentication (24-hour expiry)*

3. *Protected routes — verifyToken middleware on all private endpoints*
 4. *Role-based access — isAdmin middleware for admin-only routes*
 5. *Parameterized SQL queries — prevents SQL injection (`db.query('SELECT \* WHERE id = ?', [id])` instead of string concatenation)*
 6. *CORS — restricts which origins/methods/headers are allowed"
-

■ BONUS — How to Start Explaining (Opening Script)

When they say "Tell me about NutriGuide", say:

"NutriGuide is a full-stack web application I built for personalized Indian diet tracking. The tech stack is React on the frontend, Express + Node.js on the backend, and MySQL for the database.

The three main features are:

One — personalized meal recommendations based on the user's health goal, diet preference, and allergies.

Two — daily meal logging where users log breakfast, lunch, dinner, and snacks.

Three — weekly nutrition analytics showing calories, protein, carbs, and fat trends in charts.

Authentication is JWT-based with bcrypt password hashing, and I have role-based access control — a regular user sees the dashboard and meals, while an admin can also access the admin panel to manage recipes.

Would you like me to go deeper into any specific part — the authentication flow, database design, API structure, or the recommendation algorithm?"

■ QUICK REFERENCE CHEATSHEET